Lab 8 – Stepper Motor Control

Working with your project team, extend the stepper_async_timer_shifter.py class so that when 2 motors (axes) named azimuth (az) and altitude (alt) are commanded at the same time, they arrive at their targets at the same time, regardless of the angular distance travelled by each axis.

You are given lab8.py, which is the stepper_async_timer_shifter.py code with new parts you must write. Read the whole file before you start, particularly _tick(), _advance(), and _arm(), as well as the new public class methods move_together() and go_together(). The timing model is not yours to change: asyncio decides what to do, and the hardware timer decides when. The timer callback runs at STEP_FREQ Hz and must not allocate memory, with integer arithmetic only, no floats, no range(), no print().

In the original code, every armed motor takes a step on every tick. Two axes handed different step counts therefore start together but finish apart: the short axis runs out early and sits still while the long one catches up. Any two-axis machine (a plotter, a CNC table, or your laser turret) should be able to move in a straight line, rather than a dogleg, so the original class needs improvement.

Your job in this lab is to write a new _move_group() method that will allow multiple motors to arrive at their targets simultaneously. Before starting, set up the following hardware:

- Two 28BYJ-48 stepper motors driven through a ULN2803 Darlington array from a 74HC595 shift register (using the existing Shifter class) controlled by an ESP32-C3.
- Power the ULN2803 with 5 V.
- Call the first motor az (bits 0–3) and the second motor alt (bits 4–7).

Part A – Observe the problem

Wire the hardware and confirm the initial provided code runs before you change anything. Note what happens when commanding one motor to turn to 60° while the other turns 20°; the second motor should reach its final position in 1/3 the time of the other.

Part B – Think about how to implement an algorithm to gate the steps

The fix is to let each axis skip ticks. If an axis has to take n steps while the move as a whole lasts N ticks (where N is the step count of the longest axis), it should step on n of those N ticks, spaced as evenly as integers allow. This is a version of integer-accumulator form of the Digital Differential Analyzer (DDA) algorithm commonly used to draw a line on a pixelated 2D grid, but here one axis of the grid is time and the other is angle. Look up the algorithm to help understand how to implement it here. The provided code template includes existing methods and several new additions to make the process easier:

1. You will want to leverage the step-count math in the existing helper functions _steps_for_delta() and _steps_to_angle(), which return a (step count, direction) pair. Have your coordinated moves call these same helpers.
2. Three integer states needed for the algorithm have been provided in __init__():
   - _num the number of steps the motor must take for the current move, set from the planned step count.
   - _den the number of ticks the shared move gets, i.e. the largest step count of all motors in the group.
   - _err the accumulator. _err is the motor's running credit toward its next step, measured in units where _den buys one full step. Each tick pays in _num; once the balance reaches _den the motor has earned a step, takes it, and _den is deducted, leaving the remainder to carry forward so nothing is lost to rounding. Equivalently, _err / _den is the fractional part of the motor's ideal position: how far past its last real step the shaft should be, kept as a numerator over _den so the whole thing

stays in integer arithmetic. It's a leftover remainder that never exceeds _den, which is why the crossings come out evenly spaced and the last one lands exactly on tick _den.

3. Code for updating and evaluating _err is provided in _advance(): this code updates the accumulator for the current tick, and if no step is due, returns the chain word unchanged.
4. Note that _arm() has been updated so that a solo move sets up the states as a one-axis group via _num = _den, ensuring that the _advance() code works for single motors as well as groups.
5. Public methods move_together() and go_together() are provided as wrappers around _move_group() to allow users of the class to move multiple motors through your algorithm. Use these methods as written, and do not introduce any new methods.

Part C – Write a new _move_group() method to move multiple axes together

Write _move_group(), which is launched as a task from both of the provided public wrappers move_together() and go_together(). A call looks like:

```
await Stepper.go_together((az, 60), (alt, 20))
```

Although this lab uses only 2 motors, the Stepper class is intended to support an arbitrary number of motors, so your solution must do the same. Note that the argument lists for move_together() and go_together() already support this, with the * symbol packing an arbitrary number of arguments into a tuple, which is then passed to _move_group(). Your _move_group() code needs to be able to work with this tuple regardless of the number of elements (each corresponding to a single motor).

Write your _move_group() method to do five things, in this order:

1. Acquire every listed motor's lock. Sort the motors into a fixed order first.
2. Convert each motor's target into a step count and direction (after the locks are held).
3. Set the move length N to the largest of the step counts. Return early if every axis has nothing to do.
4. Arm every axis. This must be atomic with respect to the timer callback: use machine.disable_irq() and enable_irq() around the whole arming loop.
5. Wait for the move to finish, and release every lock you took even if something above raised an exception.

Each of those has a failure mode you need to think about:

- Lock order. Two tasks that ask for the same pair of locks in opposite orders each end up holding one and waiting forever for the other. Your fixed order should make that impossible.
- Atomic arming. If a tick lands partway through the arming loop, the axes already armed take a step and the rest do not. State in your comment how much head start that is, in milliseconds, at the default STEP_FREQ.
- A zero-length axis. go_together() may be handed an axis that is already at its target. It is never armed, so it will never signal completion — make sure your wait does not hang on it.
- Interoperation. A coordinated move must queue behind solo moves already issued on those motors, and solo moves issued afterwards must wait for it.

A detailed guide to aid with development of _move_group() is provided as an appendix below.

Part D – Demonstration

Run the demonstration code at the end of the template. Do not change the demo before validating operation and capturing video for lab submission.

Submit to Canvas:

(a) your final lab8.py file, and
(b) a short video demonstrating code operation in Part D.

All group members must submit, with all partners names entered as a submission comment.

Appendix – Step-by-step guide to writing _move_group()

_move_group() is the only method you need to write. Note that everything it needs already exists in the file: each motor has a lock (_lock), a completion flag (_done), a chain position (offset), the two step-math helpers (_steps_to_angle() and _steps_for_delta()), and the four DDA attributes (_err, _num, _den, _remaining) that the timer callback reads. Your job is to set those attributes correctly and at the right moment.

Read _tick(), _advance(), and _arm() before you start. _arm() is a single-axis version of what you are about to write for a group, and the comments in _advance() explain the DDA gate you are configuring.

The goal is for every axis to start on the same tick, finish on the same tick, and step as evenly as integers allow in between.

STEP 0 - Sanity-check the argument
`moves` is a sequence of (motor, degrees) pairs. Build a list of just the motors from it. Confirm that no motor appears twice. A duplicate would mean acquiring the same lock twice in the next step, which deadlocks the task against itself, and it would mean two conflicting step plans written to one motor. Raise a ValueError if you find one. (Objects are compared by identity here, not by equality, so compare id(m) values or use a set of ids.)

STEP 1 - Acquire every lock in a deadlock-free order
Create a list of motors sorted by their position in the chain (regardless of the order in which they were passed), and acquire locks for each. Each motor's chain position is its `offset` attribute, so sort on that. Be sure to await on lock acquisition to ensure each lock is secure before continuing. Save the locks in a `held` list so that they can be released in the same order once the shared move is complete.

Append each motor to your `held` list after its acquire() succeeds: you must release exactly the locks you actually hold. Note that you are calling `await m._lock.acquire()` here rather than using `async with`, because the number of locks is not known until runtime. That means you also need to handle the release (see Step 7).

Why sorted, not as-passed: imagine one task calls go_together((m1,..), (m2,..)) while another calls go_together((m2,..), (m1,..)) at the same instant. Taking locks in argument order, task A holds m1 and waits for m2 while task B holds m2 and waits for m1 - neither ever proceeds. A single global ordering that every caller obeys makes that impossible.

STEP 2 - Plan the move
With every lock held, convert each (motor, degrees) input into a step count and a direction, and collect the results in a list of (motor, steps, direction) triples. For an absolute move via go_together(), use the motor's own m._steps_to_angle(target), which picks the shorter way around from where that shaft is right now. For a relative move via move_together(), use cls._steps_for_delta(delta). Use the `relative` flag passed into _move_group() to decide which wrapper you call.

Why the planning happens here and not before Step 1: an absolute target is computed from the shaft's current position. If you planned first, any move still queued ahead of you would move the shaft afterward and your step count would be stale. Holding the lock means every earlier move on that axis has already finished.

STEP 3 - Choose the common time base

Find the largest step count in your plan and call it `den`. This is the number of timer ticks the whole move will take.

The `den` value is what synchronizes the axes. Each motor will be given _num = its own step count and _den = this shared total, and each motor spreads its steps over the same span of ticks. The longest axis gets _num == _den and steps on every tick, while a shorter axis skips ticks proportionally. The DDA in _advance() guarantees the final crossing lands on tick _den for all of them, which is why they finish together.

If `den` is zero, nothing needs to move at all, so return early (the `finally` in Step 7 still releases your locks).

STEP 4 - Clear the completion flags

Before arming anything, call _done.clear() on every motor in the plan. A flag left set from a previous move would make your wait in Step 5 return immediately and the method would think the move was already over.

Do this *before* the arming block, not inside it, since clear() is not something you want between disabling and enabling interrupts.

STEP 5 - Arm every axis atomically

Interrupts must be off before updating the DDA state. The timer fires at 833 Hz, so a tick can land in the middle of your loop. If it did, the motors you had already armed would take a step while the rest were still unarmed, giving them a one-tick (~1.2 ms) head start. The whole point of the method is that the axes start on the same tick. So disable interrupts, write the DDA states for all motors, then re-enable interrupts using disable_irq() / enable_irq():

```
state = disable_irq()
... write every motor's fields ...
enable_irq(state)
```

For each (motor, steps, direction) in the plan, set:

```
_err       = 0          reset the DDA accumulator
_num       = steps      this axis's own step count
_den       = den        the shared tick count from Step 3
_dir       = direction
_remaining = steps      WRITTEN LAST
```

_remaining is what arms the callback: _tick() only touches a motor whose _remaining is nonzero. Writing it last means the callback never sees a motor that is armed but whose _num/_den/_dir are still from the last move. _arm() already does this, and the same rule applies here.

Keep this block short and free of allocation - no prints, no awaits, no building lists. Nothing may yield while interrupts are disabled.

STEP 6 - Wait for completion

Await each motor's _done flag, skipping any axis whose step count was zero; that axis was never armed, so its flag will never be set and awaiting it would hang forever.

Because the DDA makes every axis land its last step on the same tick, all the flags are set at the same instant; awaiting them one after another is fine and needs no special ordering.

STEP 7 - Release the locks

Release every lock in your held list. Put the release in a `finally:` block wrapping Steps 1-6 so the locks come back even if the plan math raised, the caller cancelled the task, or the wait was interrupted. A lock leaked here silently wedges that motor for the rest of the program.

SHAPE OF THE FINISHED METHOD

```
motors = ...                          # Step 0
check for duplicates                  # Step 0
held = []
try:
    for m in sorted(...):             # Step 1
        await ...
        held.append(m)

    plans = [...]                     # Step 2
    den = ...                         # Step 3
    if den == 0: return

    clear flags                       # Step 4

    state = disable_irq()             # Step 5
    for ...: write fields
    enable_irq(state)

    for ...: await ..._done.wait()    # Step 6
finally:                              # Step 7
    for m in held:
        release
```